

Mission Crisis — Disaster Response Message Classification

Goal: Classify emergency messages into humanitarian categories to speed up disaster response.

Dataset: Figure Eight / Appen Disaster Response Messages (~30K messages from real disasters)

Target: Binary classification — is this message a request for help?

Code Input

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, VotingClassifier, StackingClassifier
)
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    f1_score, precision_score, recall_score, ConfusionMatrixDisplay, RocCurveDisplay
)
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

import xgboost as xgb
from interpret.glassbox import ExplainableBoostingClassifier
import shap
import joblib

plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette('husl')
print("All imports loaded.")
```

Output

```
All imports loaded.
```

1. Real-World Impact Framing (Q1 — 5 marks)

Item	Detail
Domain	Mission Crisis — Disaster Response
Problem	During disasters, thousands of messages flood in; manual triage is too slow
Beneficiaries	Emergency responders (Red Cross, FEMA), affected populations
Prediction Target	Binary: Is this message a request for help? (1=Yes, 0=No)
Measurable Impact	Reduce message triage time from hours to seconds
Why ML?	Volume exceeds human capacity; text patterns reliably signal urgency
Dataset	Figure Eight/Appen Disaster Response Messages (Kaggle, public domain)
Unit of Analysis	Individual emergency message
Limitations	English-biased; may not generalise to unseen disaster types; needs human oversight

Code Input

```
import os

# Load dataset from local data/ directory (downloaded from GitHub mirror of Figure Eight dataset)
path = os.path.join(os.getcwd(), 'data')
print(f"Dataset path: {path}")
for f in os.listdir(path):
    print(f"  {f}")
```

Output

```
Dataset path: /Users/dan/projects/pythonVishal/machine_learning/cia3/data
disaster_messages.csv
disaster_categories.csv
```

Code Input

```

messages = pd.read_csv(os.path.join(path, 'disaster_messages.csv'))
categories = pd.read_csv(os.path.join(path, 'disaster_categories.csv'))
print(f"Messages shape: {messages.shape}")
print(f"Categories shape: {categories.shape}")

# Merge on 'id'
df = messages.merge(categories, on='id', how='inner')
print(f"Merged shape: {df.shape}")
df.head()

```

Output

```

Messages shape: (26248, 4)
Categories shape: (26248, 2)
Merged shape: (26386, 5)

```

Result

```

id                                message \
0    2  Weather update - a cold front from Cuba that c...
1    7                Is the Hurricane over or is it not over
2    8                Looking for someone but no name
3    9  UN reports Leogane 80-90 destroyed. Only Hospi...
4   12  says: west side of Haiti, rest of the country ...

                                original  genre \
0  Un front froid se retrouve sur Cuba ce matin. ... direct
1                Cyclone nan fini osinon li pa fini direct
2  Patnm, di Maryani relem pou li banm nouvel li ... direct
3  UN reports Leogane 80-90 destroyed. Only Hospi... direct
4  facade ouest d Haiti et le reste du pays aujou... direct

                                categories
0  related-1;request-0;offer-0;aid_related-0;medi...
1  related-1;request-0;offer-0;aid_related-1;medi...
2  related-1;request-0;offer-0;aid_related-0;medi...
3  related-1;request-1;offer-0;aid_related-1;medi...
4  related-1;request-0;offer-0;aid_related-0;medi...

```

Code Input

```
# Categories column has format: "related-1;request-0;offer-0;..."
cat_cols = df['categories'].str.split(';', expand=True)

# Extract column names from first row
cat_names = [c.split('-')[0] for c in cat_cols.iloc[0]]
cat_cols.columns = cat_names

# Convert to numeric (take digit after dash)
for col in cat_names:
    cat_cols[col] = cat_cols[col].str.split('-').str[-1].astype(int)

# Fix any values > 1 (data quality issue – cap at 1)
cat_cols = cat_cols.clip(upper=1)

df = pd.concat([df.drop('categories', axis=1), cat_cols], axis=1)
print(f"Final shape: {df.shape}")
print(f"Category columns: {len(cat_names)}")
df.head()
```

Output

```
Final shape: (26386, 40)
Category columns: 36
```

Result

```
id          message \
0   2  Weather update – a cold front from Cuba that c...
1   7           Is the Hurricane over or is it not over
2   8           Looking for someone but no name
3   9  UN reports Leogane 80-90 destroyed. Only Hospi...
4  12  says: west side of Haiti, rest of the country ...

          original  genre  related \
0  Un front froid se retrouve sur Cuba ce matin. ...  direct      1
1          Cyclone nan fini osinon li pa fini  direct      1
2  Patnm, di Maryani relem pou li banm nouvel li ...  direct      1
3  UN reports Leogane 80-90 destroyed. Only Hospi...  direct      1
4  facade ouest d Haiti et le reste du pays aujou...  direct      1

request  offer  aid_related  medical_help  medical_products  ... \
0         0     0           0             0                 0 ...
1         0     0           1             0                 0 ...
2         0     0           0             0                 0 ...
3         1     0           1             0                 1 ...
4         0     0           0             0                 0 ...

aid_centers  other_infrastructure  weather_related  floods  storm  fire \
0           0                   0                 0     0     0     0
1           0                   0                 1     0     1     0
2           0                   0                 0     0     0     0
3           0                   0                 0     0     0     0
4           0                   0                 0     0     0     0

earthquake  cold  other_weather  direct_report
0           0     0             0             0
1           0     0             0             0
2           0     0             0             0
3           0     0             0             0
```

4	0	0	0	0
[5 rows x 40 columns]				

2. Data Wrangling & Feature Engineering (Q2 — 6 marks)

Code Input

```
print("=== Data Types ===")
print(df.dtypes)
print(f"\n=== Missing Values ===")
print(df.isnull().sum()[df.isnull().sum() > 0])
print(f"\n=== Duplicates: {df.duplicated().sum()}")
print(f"=== Shape: {df.shape}")
```

Output

```
=== Data Types ===
id                int64
message           str
original          str
genre             str
related           int64
request           int64
offer             int64
aid_related       int64
medical_help      int64
medical_products  int64
search_and_rescue int64
security          int64
military          int64
child_alone       int64
water             int64
food              int64
shelter           int64
clothing          int64
money             int64
missing_people    int64
refugees          int64
death             int64
other_aid         int64
infrastructure_related int64
transport         int64
buildings         int64
electricity       int64
tools             int64
hospitals         int64
shops             int64
aid_centers       int64
other_infrastructure int64
weather_related   int64
floods            int64
storm             int64
fire              int64
earthquake        int64
cold              int64
other_weather     int64
direct_report     int64
dtype: object

=== Missing Values ===
original    16140
dtype: int64

=== Duplicates: 171
=== Shape: (26386, 40)
```

Code Input

```
# Drop rows with missing messages (can't classify without text)
df = df.dropna(subset=['message'])

# Remove exact duplicates
df = df.drop_duplicates(subset=['message'])
print(f"After cleaning: {df.shape}")
```

Output

```
After cleaning: (26177, 40)
```

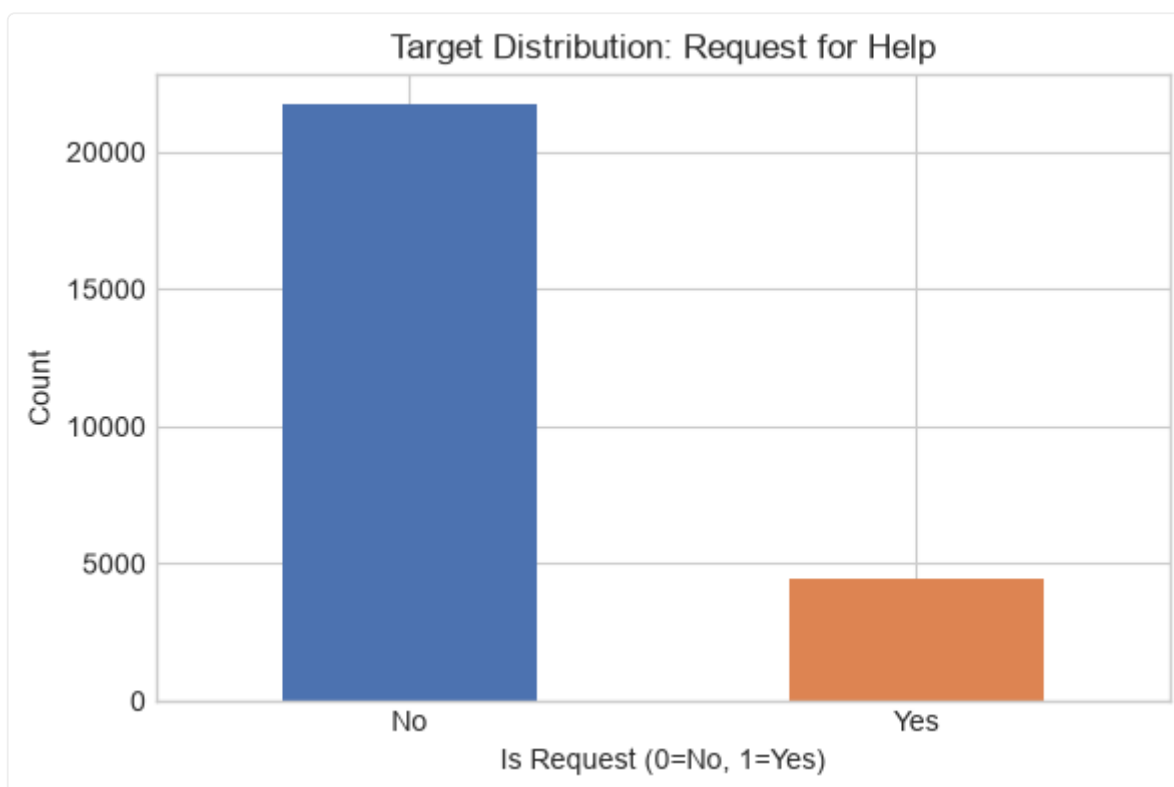
Code Input

```
target = 'request'
print(f"Target distribution:\n{df[target].value_counts()}")
print(f"Class balance: {df[target].mean():.3f} positive rate")

fig, ax = plt.subplots(figsize=(6, 4))
df[target].value_counts().plot(kind='bar', ax=ax, color=['#4C72B0', '#DD8452'])
ax.set_title('Target Distribution: Request for Help')
ax.set_xlabel('Is Request (0=No, 1=Yes)')
ax.set_ylabel('Count')
ax.set_xticklabels(['No', 'Yes'], rotation=0)
plt.tight_layout()
plt.show()
```

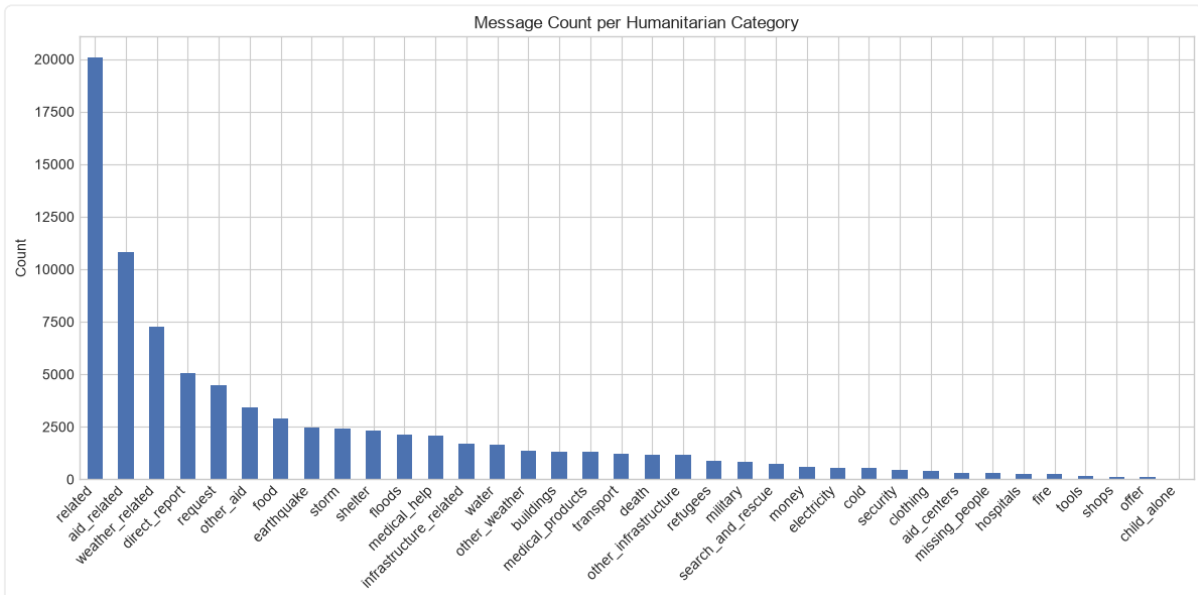
Output

```
Target distribution:
request
0    21713
1     4464
Name: count, dtype: int64
Class balance: 0.171 positive rate
```



Code Input

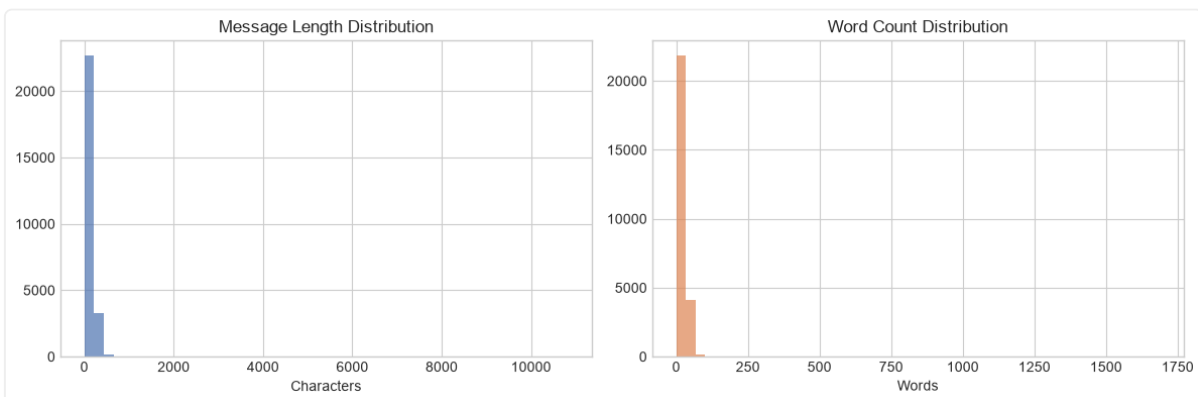
```
cat_counts = df[cat_names].sum().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(12, 6))
cat_counts.plot(kind='bar', ax=ax, color='#4C72B0')
ax.set_title('Message Count per Humanitarian Category')
ax.set_ylabel('Count')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Code Input

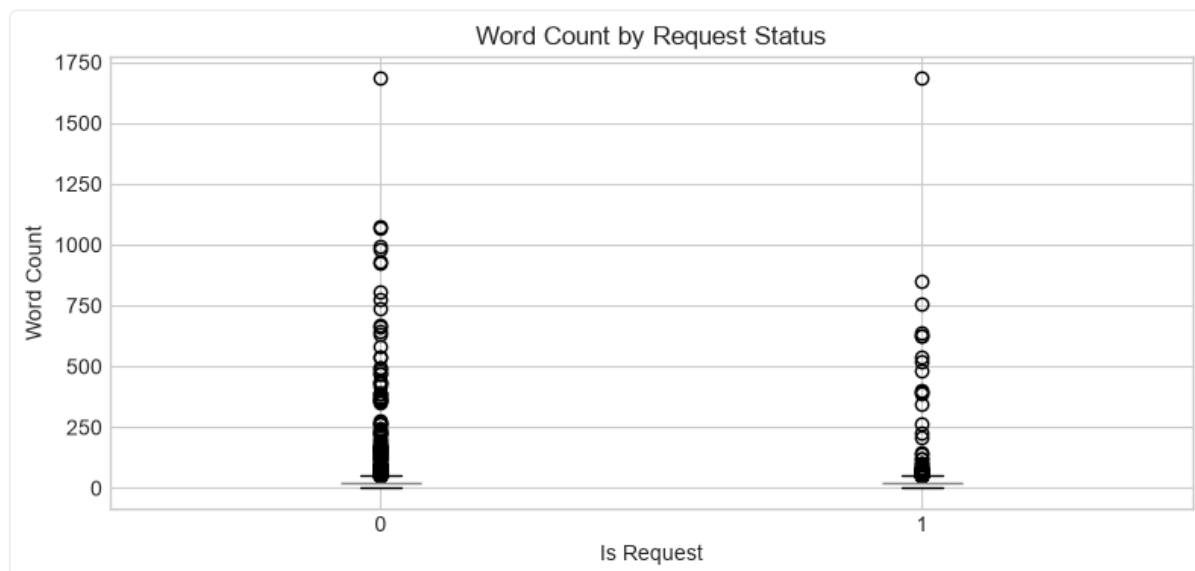
```
df['msg_len'] = df['message'].str.len()
df['word_count'] = df['message'].str.split().str.len()

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df['msg_len'].hist(bins=50, ax=axes[0], color='#4C72B0', alpha=0.7)
axes[0].set_title('Message Length Distribution')
axes[0].set_xlabel('Characters')
df['word_count'].hist(bins=50, ax=axes[1], color='#DD8452', alpha=0.7)
axes[1].set_title('Word Count Distribution')
axes[1].set_xlabel('Words')
plt.tight_layout()
plt.show()
```



Code Input

```
fig, ax = plt.subplots(figsize=(8, 4))
df.boxplot(column='word_count', by=target, ax=ax)
ax.set_title('Word Count by Request Status')
ax.set_xlabel('Is Request')
ax.set_ylabel('Word Count')
plt.suptitle('')
plt.tight_layout()
plt.show()
```



Code Input

```
# Text features: TF-IDF on message text
# Metadata features: message length, word count, genre encoding

# Genre is a categorical variable (direct, news, social)
print(f"Genre distribution:\n{df['genre'].value_counts()}")
```

Output

```
Genre distribution:
genre
news      13036
direct    10747
social     2394
Name: count, dtype: int64
```

Code Input

```
X_text = df['message'].fillna('') # text column for TF-IDF
X_meta = df[['msg_len', 'word_count']].copy()
X_meta['genre_direct'] = (df['genre'] == 'direct').astype(int)
X_meta['genre_news'] = (df['genre'] == 'news').astype(int)
X_meta['genre_social'] = (df['genre'] == 'social').astype(int)

# Engineered features: contains question mark, exclamation, uppercase ratio
X_meta['has_question'] = df['message'].str.contains(r'\?', na=False).astype(int)
X_meta['has_exclaim'] = df['message'].str.contains('!', na=False).astype(int)
X_meta['upper_ratio'] = df['message'].apply(
    lambda x: sum(1 for c in str(x) if c.isupper()) / max(len(str(x)), 1)
)

y = df[target].values
print(f"Features ready. Text: {X_text.shape[0]}, Meta: {X_meta.shape}")
```

Output

```
Features ready. Text: 26177, Meta: (26177, 8)
```

Code Input

```
X_text_train, X_text_test, X_meta_train, X_meta_test, y_train, y_test = train_test_split(
    X_text, X_meta, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Train: {len(y_train)}, Test: {len(y_test)}")
print(f"Train positive rate: {y_train.mean():.3f}")
print(f"Test positive rate: {y_test.mean():.3f}")
```

Output

```
Train: 20941, Test: 5236
Train positive rate: 0.171
Test positive rate: 0.171
```

Code Input

```
tfidf = TfidfVectorizer(max_features=1000, stop_words='english', ngram_range=(1, 2))
X_tfidf_train = tfidf.fit_transform(X_text_train) # fit on train only
X_tfidf_test = tfidf.transform(X_text_test)      # transform test

print(f"TF-IDF train shape: {X_tfidf_train.shape}")
print(f"TF-IDF test shape: {X_tfidf_test.shape}")
```

Output

```
TF-IDF train shape: (20941, 1000)
TF-IDF test shape: (5236, 1000)
```

Code Input

```
scaler = StandardScaler()
X_meta_train_scaled = scaler.fit_transform(X_meta_train)
X_meta_test_scaled = scaler.transform(X_meta_test)
```

Code Input

```
from scipy.sparse import hstack, csr_matrix

X_train = hstack([X_tfidf_train, csr_matrix(X_meta_train_scaled)])
X_test = hstack([X_tfidf_test, csr_matrix(X_meta_test_scaled)])
print(f"Combined train: {X_train.shape}")
print(f"Combined test: {X_test.shape}")
```

Output

```
Combined train: (20941, 1008)
Combined test: (5236, 1008)
```

Code Input

```
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
print(f"After SMOTE - Train: {Counter(y_train_res)}")
```

Output

```
After SMOTE - Train: Counter({np.int64(1): 17370, np.int64(0): 17370})
```

3. Ensemble Architecture, Tuning & Comparison (Q3 — 8 marks)

Code Input

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = {} # store model results

def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    """Train, predict, and store metrics."""
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[:, 1] if hasattr(model, 'predict_proba') else None

    f1 = f1_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred)
    rec = recall_score(y_te, y_pred)
    auc = roc_auc_score(y_te, y_proba) if y_proba is not None else None

    results[name] = {'F1': f1, 'Precision': prec, 'Recall': rec, 'ROC-AUC': auc}
    print(f"\n{'='*50}")
    print(f"{name}")
    print(f"{'='*50}")
    print(f"F1: {f1:.4f} | Precision: {prec:.4f} | Recall: {rec:.4f} | AUC: {auc:.4f}" if auc else
f"F1: {f1:.4f}")
    print(classification_report(y_te, y_pred, target_names=['Not Request', 'Request']))

    # Confusion matrix
    fig, ax = plt.subplots(figsize=(5, 4))
    ConfusionMatrixDisplay.from_predictions(y_te, y_pred, display_labels=['Not Request', 'Request'],
ax=ax, cmap='Blues')
    ax.set_title(f'{name} — Confusion Matrix')
    plt.tight_layout()
    plt.show()

    return model
```

Code Input

```
print("Training baseline...")
lr = LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced')
lr_cv = cross_val_score(lr, X_train_res, y_train_res, cv=cv, scoring='f1')
print(f"Logistic Regression CV F1: {lr_cv.mean():.4f} (+/- {lr_cv.std():.4f})")
lr_model = evaluate_model('Logistic Regression (Baseline)', lr, X_train_res, y_train_res, X_test,
y_test)
```

Output

Training baseline...

Output

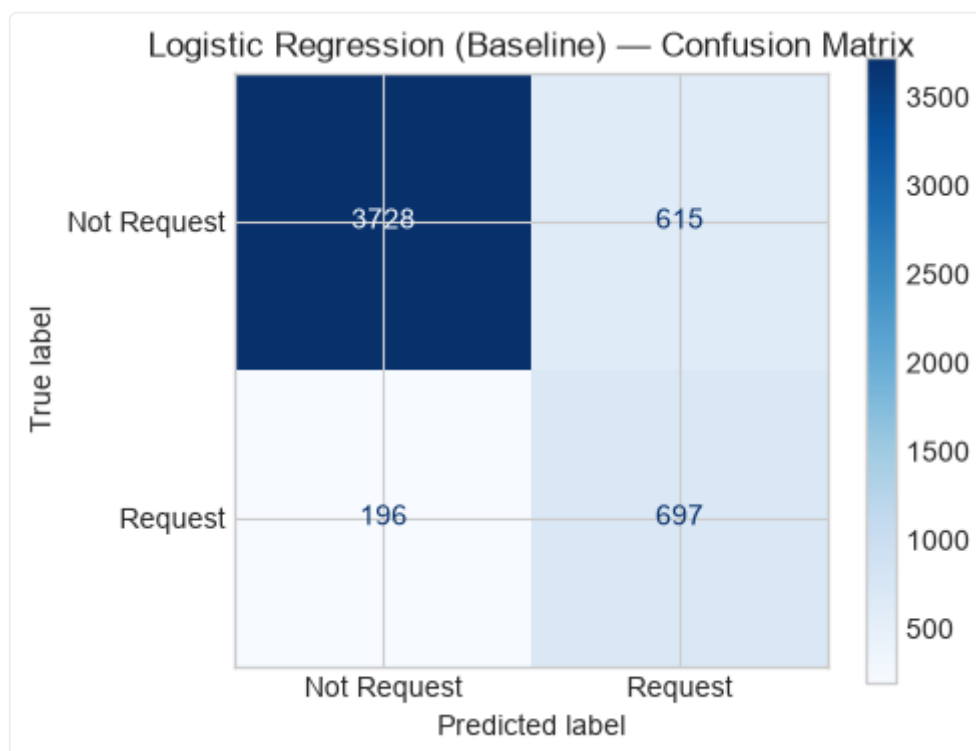
Logistic Regression CV F1: 0.8373 (+/- 0.0028)

Output

```
=====
Logistic Regression (Baseline)
=====
F1: 0.6322 | Precision: 0.5312 | Recall: 0.7805 | AUC: 0.8887
precision    recall  f1-score   support

Not Request    0.95    0.86    0.90    4343
Request        0.53    0.78    0.63     893

accuracy              0.85    5236
macro avg    0.74    0.82    0.77    5236
weighted avg    0.88    0.85    0.86    5236
```



Code Input

```
print("Training Random Forest...")
rf = RandomForestClassifier(n_estimators=100, max_depth=25, random_state=42, n_jobs=-1,
                           class_weight='balanced')
rf_cv = cross_val_score(rf, X_train_res, y_train_res, cv=cv, scoring='f1', n_jobs=-1)
print(f"Random Forest CV F1: {rf_cv.mean():.4f} (+/- {rf_cv.std():.4f})")
rf_model = evaluate_model('Random Forest (Bagging)', rf, X_train_res, y_train_res, X_test, y_test)
```

Output

Training Random Forest...

Output

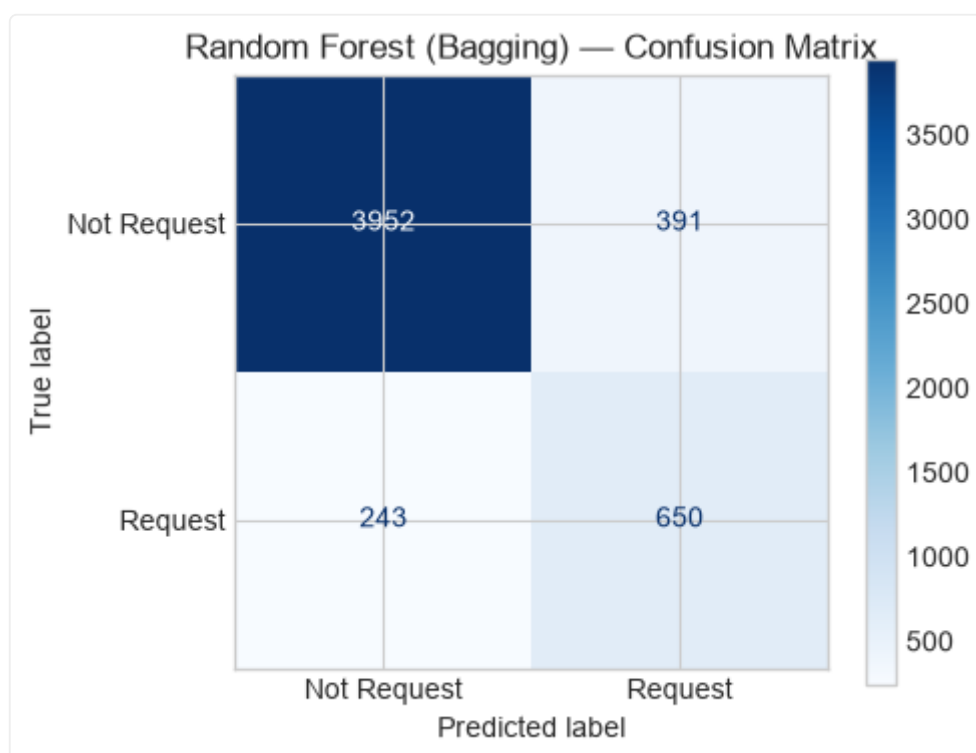
Random Forest CV F1: 0.8921 (+/- 0.0030)

Output

```
=====
Random Forest (Bagging)
=====
F1: 0.6722 | Precision: 0.6244 | Recall: 0.7279 | AUC: 0.8804
precision    recall  f1-score   support

Not Request    0.94    0.91    0.93     4343
Request        0.62    0.73    0.67       893

accuracy              0.88     5236
macro avg    0.78    0.82    0.80     5236
weighted avg    0.89    0.88    0.88     5236
```



Code Input

```
print("Training XGBoost...")
xgb_clf = xgb.XGBClassifier(
    n_estimators=150, max_depth=5, learning_rate=0.1,
    scale_pos_weight=(y_train == 0).sum() / (y_train == 1).sum(),
    random_state=42, eval_metric='logloss', n_jobs=-1
)
xgb_cv = cross_val_score(xgb_clf, X_train_res, y_train_res, cv=cv, scoring='f1', n_jobs=-1)
print(f"XGBoost CV F1: {xgb_cv.mean():.4f} (+/- {xgb_cv.std():.4f})")
xgb_model = evaluate_model('XGBoost (Boosting)', xgb_clf, X_train_res, y_train_res, X_test, y_test)
```

Output

Training XGBoost...

Output

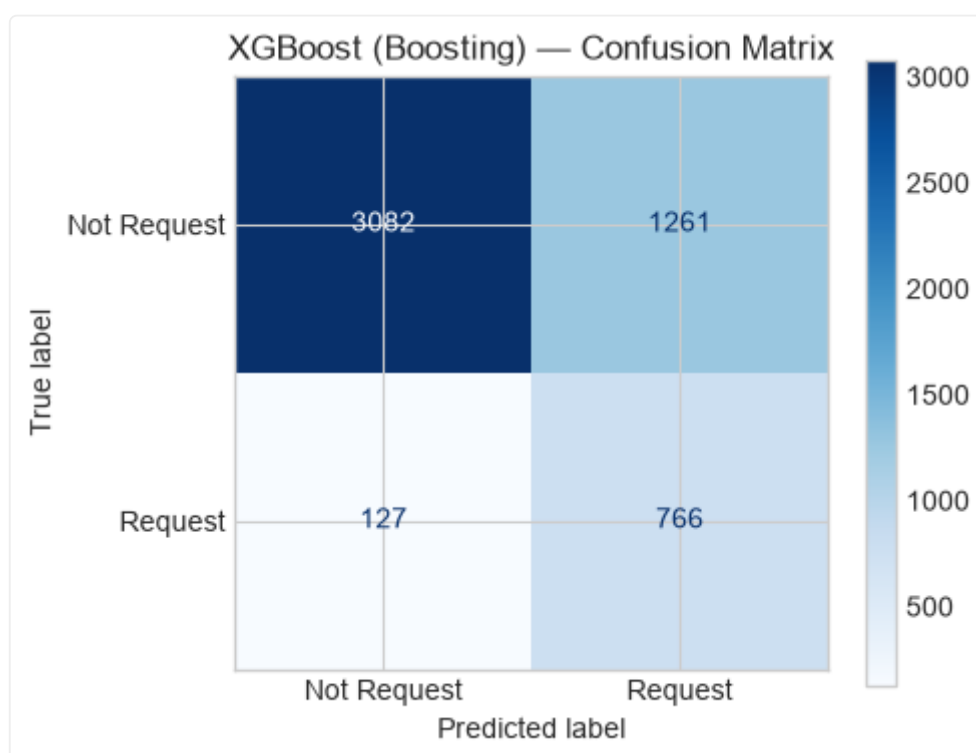
XGBoost CV F1: 0.8570 (+/- 0.0037)

Output

```
=====
XGBoost (Boosting)
=====
F1: 0.5247 | Precision: 0.3779 | Recall: 0.8578 | AUC: 0.8727
      precision    recall  f1-score   support

Not Request      0.96      0.71      0.82     4343
Request          0.38      0.86      0.52      893

   accuracy
macro avg   0.67      0.78      0.67     5236
weighted avg 0.86      0.73      0.77     5236
```



Code Input

```

print("Training EBM (glass-box model)...")
from sklearn.feature_selection import SelectKBest, f_classif
# Select top 30 most discriminatory features for clear glass-box interpretation using ANOVA F-value
selector = SelectKBest(f_classif, k=min(30, X_train_res.shape[1]))
# Subsample training data for fast EBM spline fitting (5000 samples is ideal for Generalized
Additive Models)
np.random.seed(42)
sample_sub_idx = np.random.choice(X_train_res.shape[0], size=min(5000, X_train_res.shape[0]),
replace=False)
X_tr_ebm_sparse = selector.fit_transform(X_train_res[sample_sub_idx], y_train_res[sample_sub_idx])
X_te_ebm_sparse = selector.transform(X_test)

X_tr_ebm = X_tr_ebm_sparse.toarray()
X_te_ebm = X_te_ebm_sparse.toarray()

ebm_feat_indices = selector.get_support(indices=True)
tfidf_names = tfidf.get_feature_names_out().tolist()
meta_names = list(X_meta.columns)
all_feat_names_raw = tfidf_names + meta_names
ebm_feature_names = [all_feat_names_raw[i] for i in ebm_feat_indices]

ebm = ExplainableBoostingClassifier(feature_names=ebm_feature_names, interactions=0, outer_bags=2,
inner_bags=0, random_state=42, n_jobs=-1)
ebm_model = evaluate_model('EBM (Glass-Box)', ebm, X_tr_ebm, y_train_res[sample_sub_idx], X_te_ebm,
y_test)

```

Output

Training EBM (glass-box model)...

Output

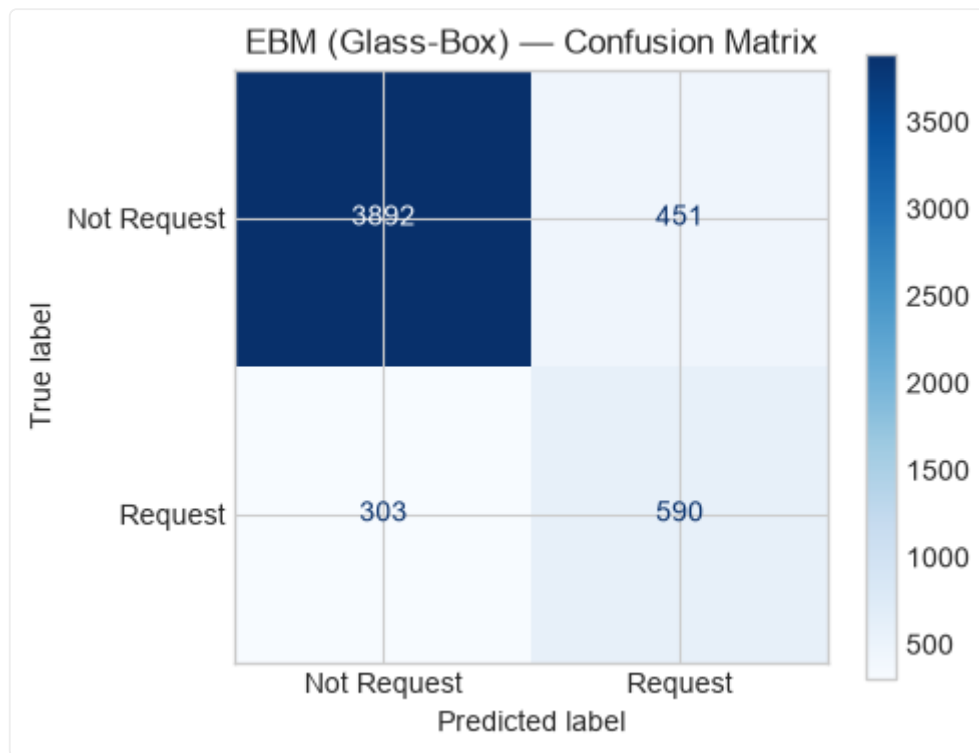
```

=====
EBM (Glass-Box)
=====
F1: 0.6101 | Precision: 0.5668 | Recall: 0.6607 | AUC: 0.8479
      precision    recall  f1-score   support

Not Request      0.93      0.90      0.91      4343
Request          0.57      0.66      0.61       893

   accuracy            0.86            5236
  macro avg      0.75      0.78      0.76      5236
weighted avg      0.87      0.86      0.86      5236

```



Code Input

```
print("Training Stacking Classifier...")
stacking = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(max_iter=300, random_state=42, class_weight='balanced')),
        ('rf', RandomForestClassifier(n_estimators=30, max_depth=12, random_state=42, n_jobs=-1,
class_weight='balanced')),
        ('xgb', xgb.XGBClassifier(n_estimators=30, max_depth=4, random_state=42,
eval_metric='logloss', n_jobs=-1)),
    ],
    final_estimator=LogisticRegression(max_iter=300, random_state=42),
    cv=3, n_jobs=-1
)
stacking_model = evaluate_model('Stacking Classifier', stacking, X_train_res, y_train_res, X_test,
y_test)
```

Output

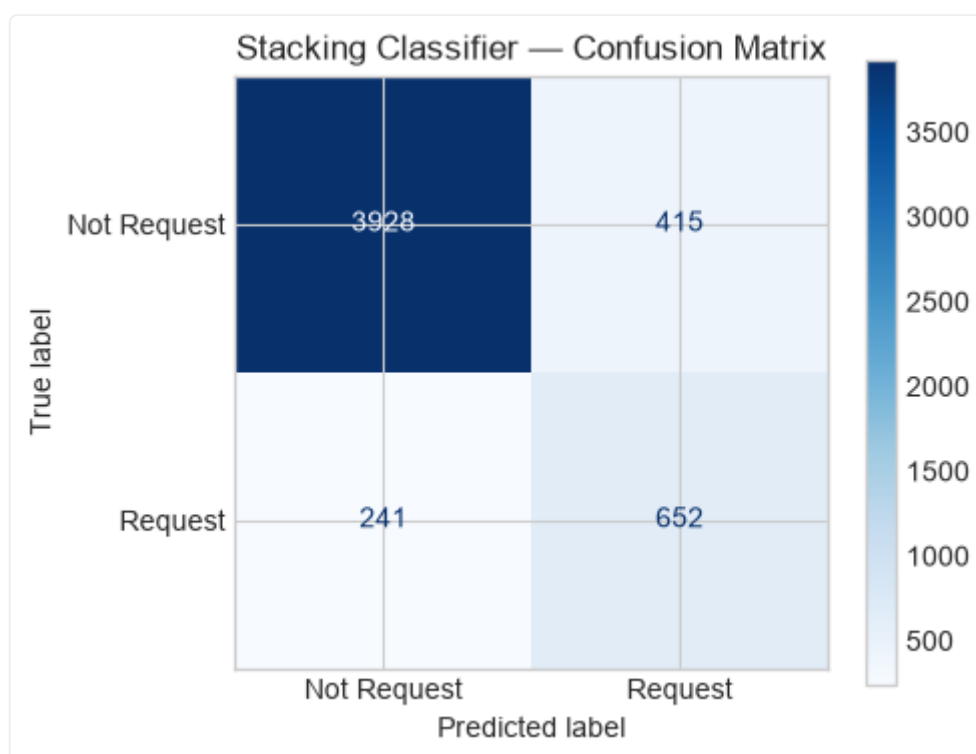
Training Stacking Classifier...

Output

```
=====
Stacking Classifier
=====
F1: 0.6653 | Precision: 0.6111 | Recall: 0.7301 | AUC: 0.8886
      precision    recall  f1-score   support

Not Request      0.94      0.90      0.92     4343
Request          0.61      0.73      0.67      893

   accuracy      0.87     5236
  macro avg      0.78      5236
weighted avg      0.89      5236
```



Code Input

```
print("Training Voting Classifier...")
voting = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(max_iter=300, random_state=42, class_weight='balanced')),
        ('rf', RandomForestClassifier(n_estimators=30, max_depth=12, random_state=42, n_jobs=-1,
class_weight='balanced')),
        ('xgb', xgb.XGBClassifier(n_estimators=30, max_depth=4, random_state=42,
eval_metric='logloss', n_jobs=-1)),
    ],
    voting='soft', n_jobs=-1
)
voting_model = evaluate_model('Voting Classifier', voting, X_train_res, y_train_res, X_test, y_test)
```

Output

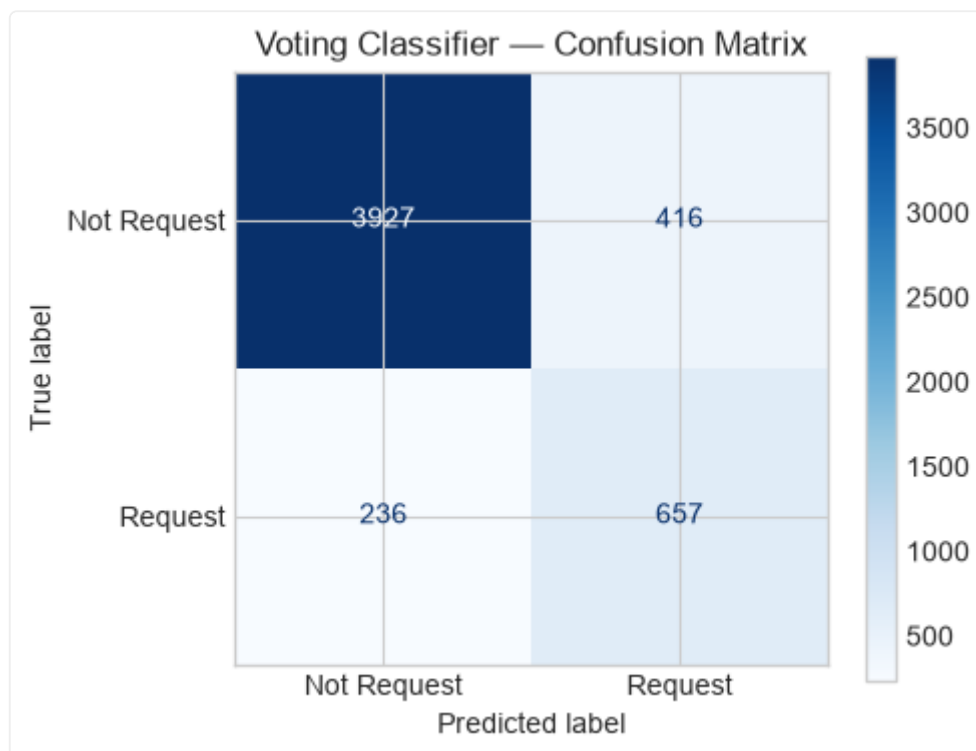
Training Voting Classifier...

Output

```
=====
Voting Classifier
=====
F1: 0.6684 | Precision: 0.6123 | Recall: 0.7357 | AUC: 0.8886
      precision    recall  f1-score   support

Not Request      0.94      0.90      0.92     4343
Request          0.61      0.74      0.67      893

   accuracy      0.88     5236
  macro avg      0.78     5236
weighted avg      0.89     5236
```



Code Input

```

results_df = pd.DataFrame(results).T.round(4)
results_df = results_df.sort_values('F1', ascending=False)
print("\n" + "="*60)
print("MODEL COMPARISON SUMMARY")
print("="*60)
print(results_df.to_string())

```

Output

```

=====
MODEL COMPARISON SUMMARY
=====

```

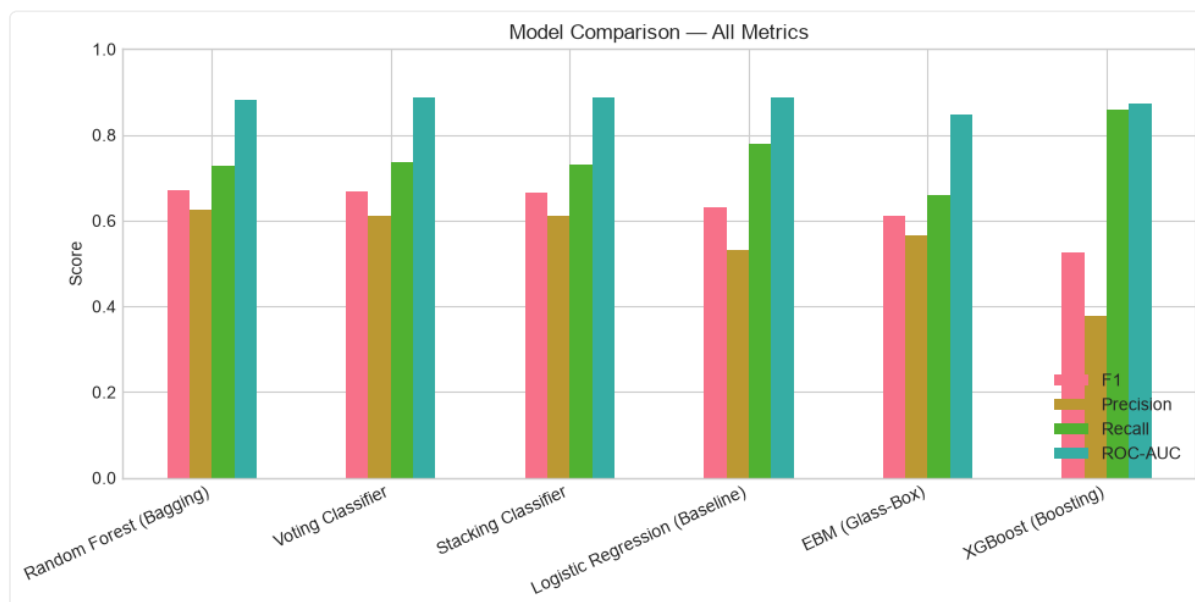
	F1	Precision	Recall	ROC-AUC
Random Forest (Bagging)	0.6722	0.6244	0.7279	0.8804
Voting Classifier	0.6684	0.6123	0.7357	0.8886
Stacking Classifier	0.6653	0.6111	0.7301	0.8886
Logistic Regression (Baseline)	0.6322	0.5312	0.7805	0.8887
EBM (Glass-Box)	0.6101	0.5668	0.6607	0.8479
XGBoost (Boosting)	0.5247	0.3779	0.8578	0.8727

Code Input

```

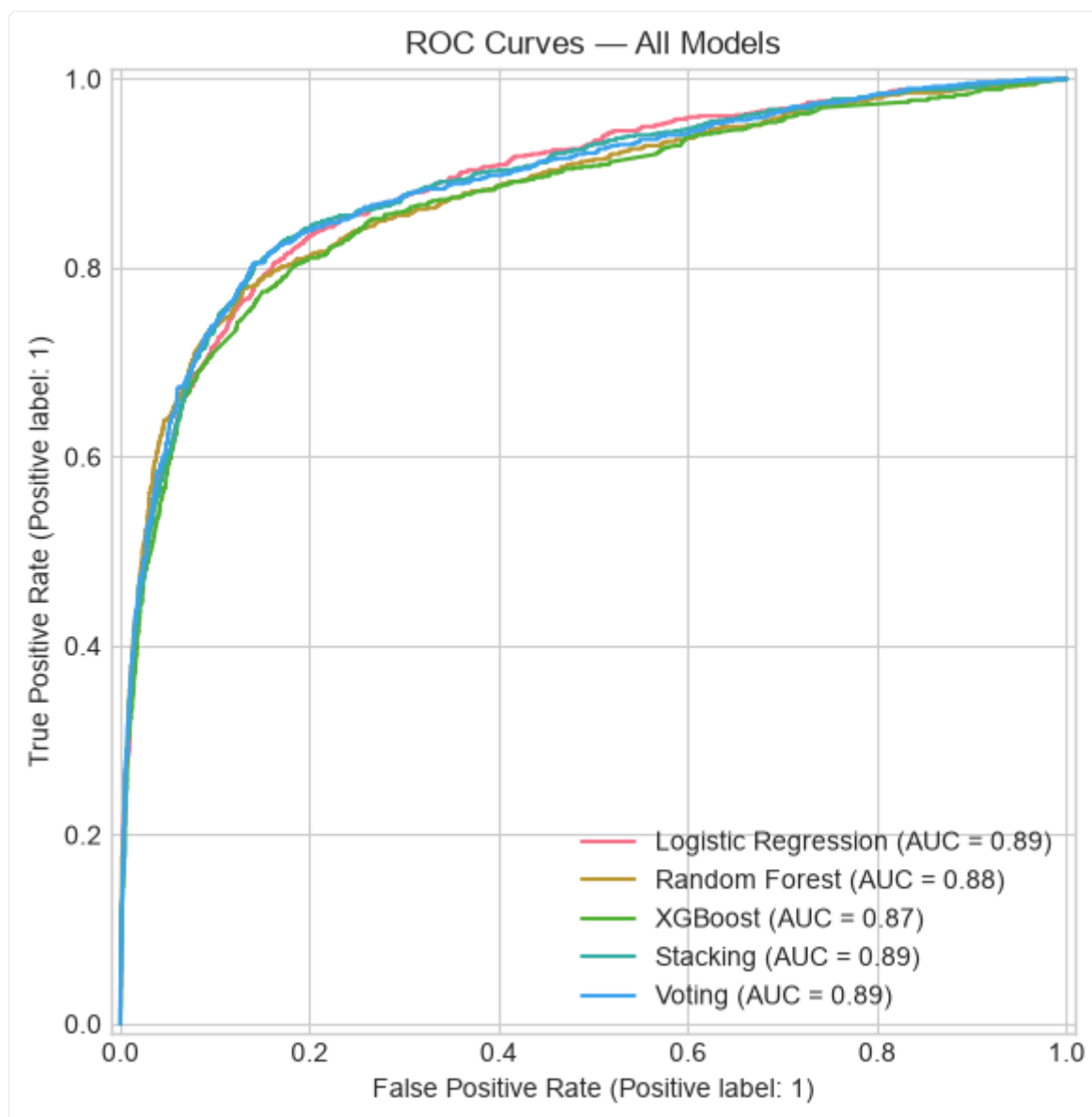
fig, ax = plt.subplots(figsize=(10, 5))
results_df[['F1', 'Precision', 'Recall', 'ROC-AUC']].plot(kind='bar', ax=ax)
ax.set_title('Model Comparison — All Metrics')
ax.set_ylabel('Score')
ax.set_ylim(0, 1)
plt.xticks(rotation=25, ha='right')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

```



Code Input

```
fig, ax = plt.subplots(figsize=(8, 6))
for name, model in [('Logistic Regression', lr_model), ('Random Forest', rf_model),
                    ('XGBoost', xgb_model), ('Stacking', stacking_model), ('Voting',
voting_model)]:
    if hasattr(model, 'predict_proba'):
        RocCurveDisplay.from_estimator(model, X_test, y_test, ax=ax, name=name)
ax.set_title('ROC Curves — All Models')
plt.tight_layout()
plt.show()
```



4. Model Explainability & Ethics (Q4 — 4 marks)

Code Input

```
print("Computing SHAP values for XGBoost...")
X_shap = X_test[:100].toarray() if hasattr(X_test, 'toarray') else X_test[:100]
tfidf_names = tfidf.get_feature_names_out().tolist()
meta_names = list(X_meta.columns)
all_feature_names = tfidf_names + meta_names

explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_shap)
```

Output

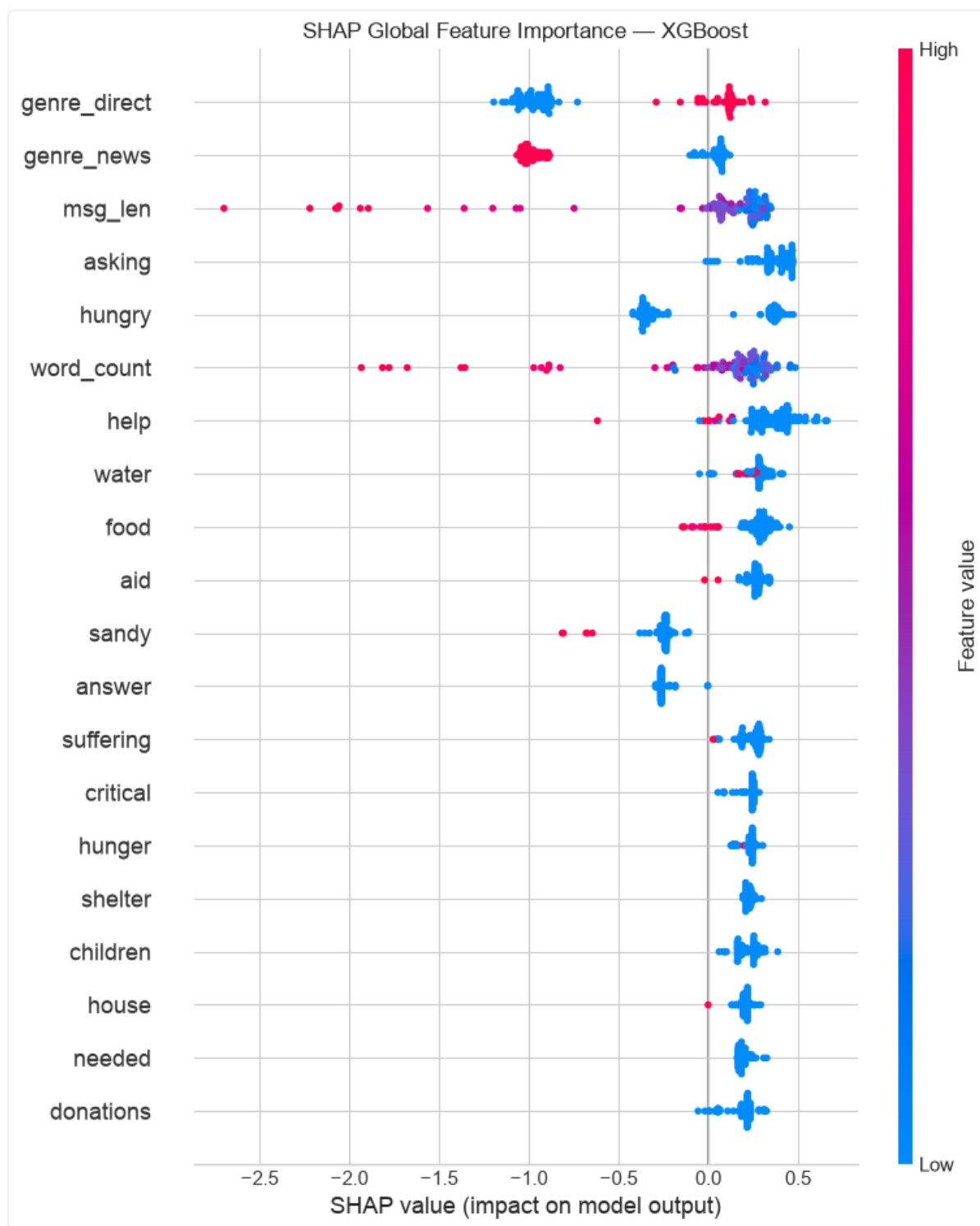
```
Computing SHAP values for XGBoost...
```

Code Input

```
print("SHAP Global Feature Importance (top 20):")
fig, ax = plt.subplots(figsize=(10, 8))
shap.summary_plot(shap_values, X_shap, feature_names=all_feature_names, max_display=20, show=False)
plt.title('SHAP Global Feature Importance — XGBoost')
plt.tight_layout()
plt.show()
```

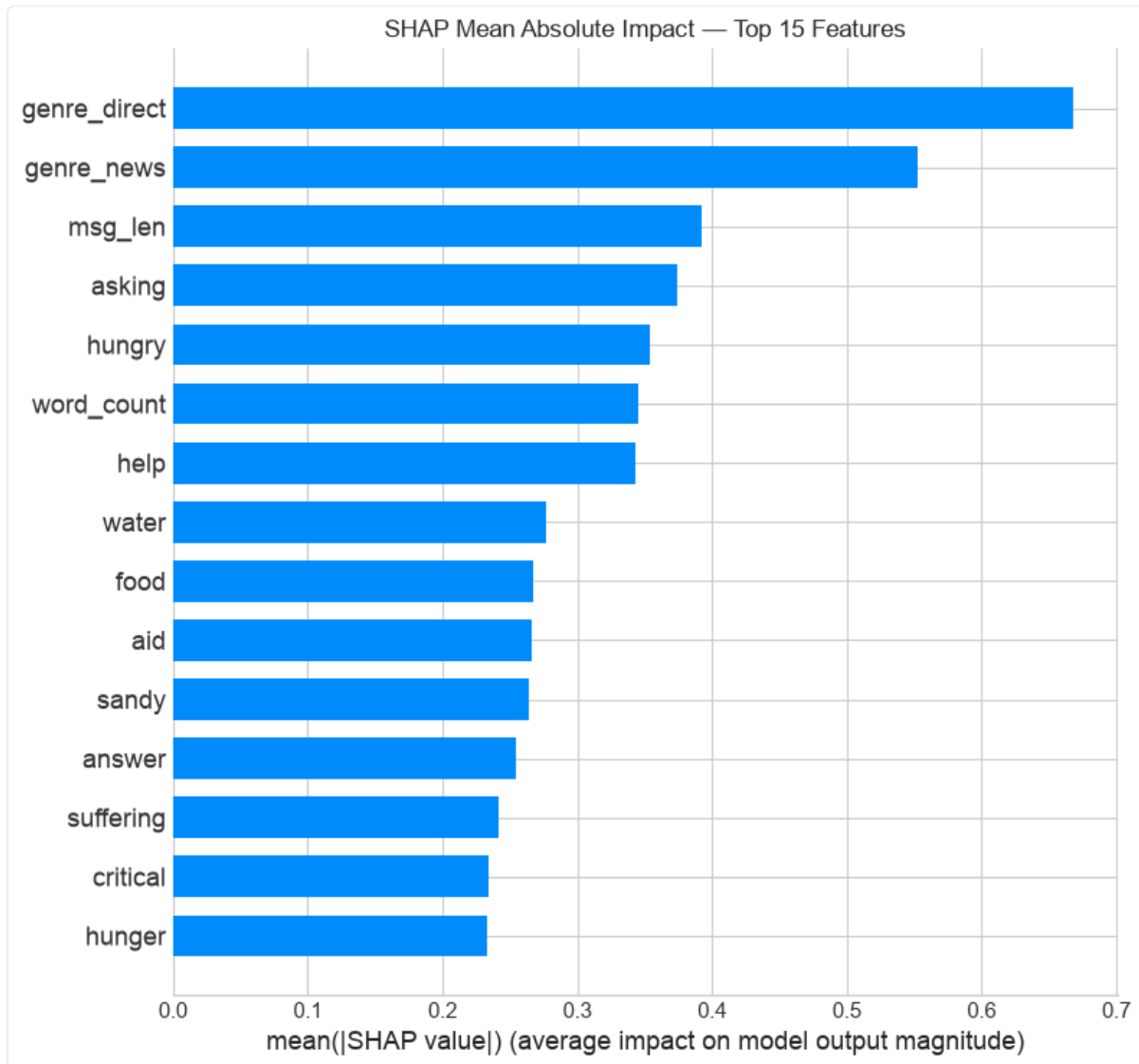
Output

SHAP Global Feature Importance (top 20):



Code Input

```
fig, ax = plt.subplots(figsize=(10, 6))
shap.summary_plot(shap_values, X_shap, feature_names=all_feature_names, plot_type='bar',
max_display=15, show=False)
plt.title('SHAP Mean Absolute Impact — Top 15 Features')
plt.tight_layout()
plt.show()
```



Code Input

```

sample_idx = 0 # first test sample
print(f"\nSample message: {X_text_test.iloc[sample_idx][:200]}...")
print(f"True label: {'Request' if y_test[sample_idx] == 1 else 'Not Request'}")
print(f"Predicted: {'Request' if xgb_model.predict(X_test[sample_idx])[0] == 1 else 'Not Request'}")

shap.initjs()
shap.force_plot(explainer.expected_value, shap_values[sample_idx],
                feature_names=all_feature_names, matplotlib=True)
plt.title('SHAP Local Explanation – Single Prediction')
plt.tight_layout()
plt.show()

```

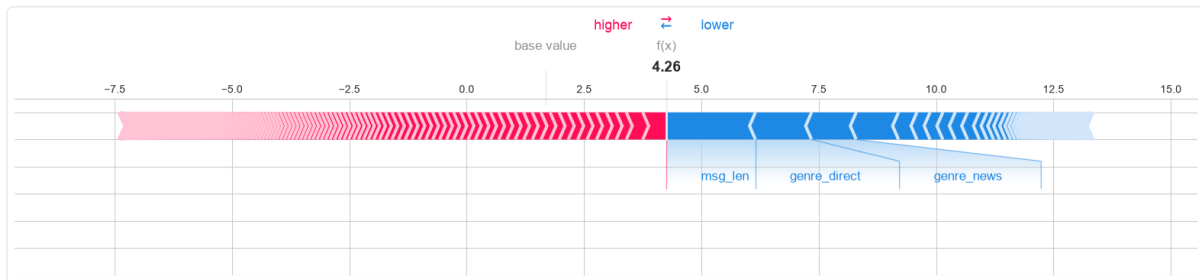
Output

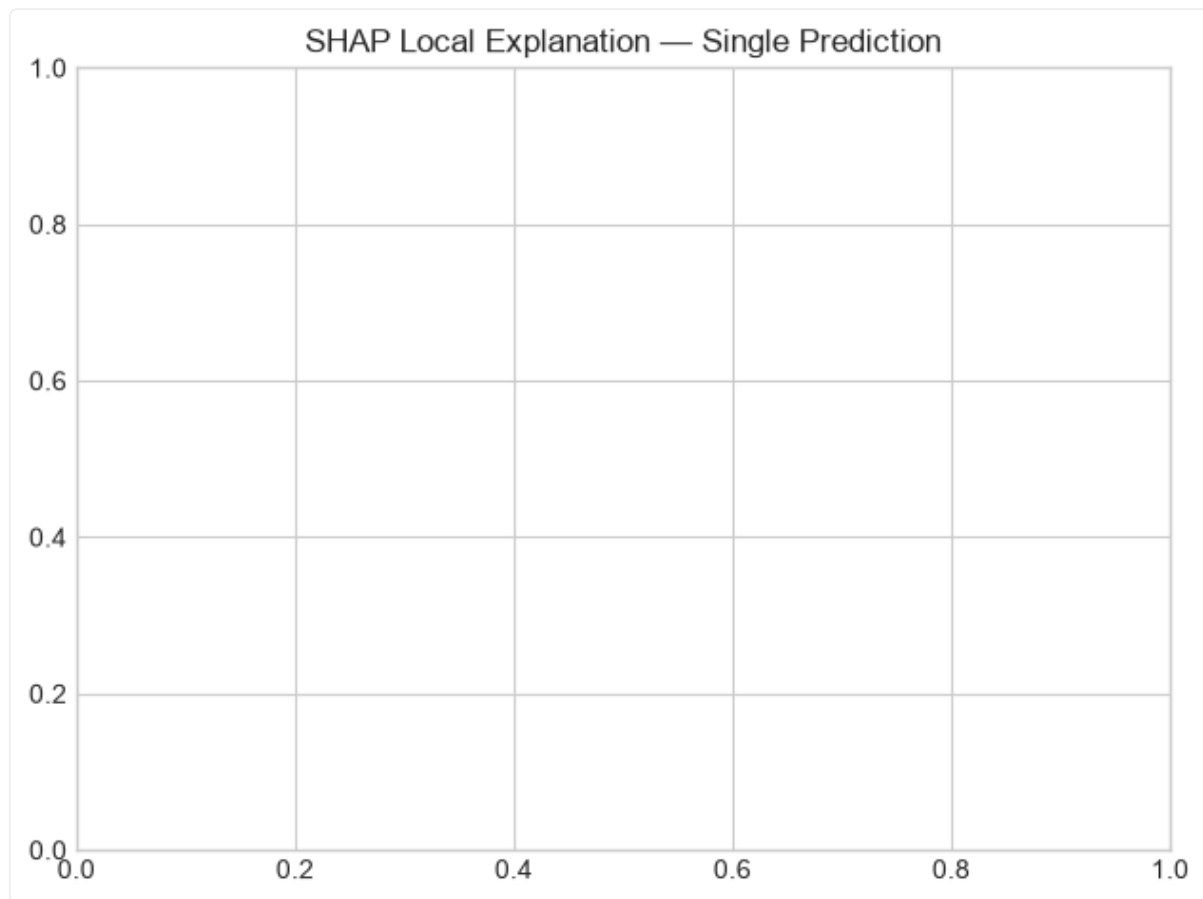
Sample message: The medical team, composed of general surgeons, internists, infectious disease specialists, pediatricians, sanitary engineers, psycho-social interventionists and paramedics from the Department of Heal...

True label: Not Request

Predicted: Not Request

Result





Code Input

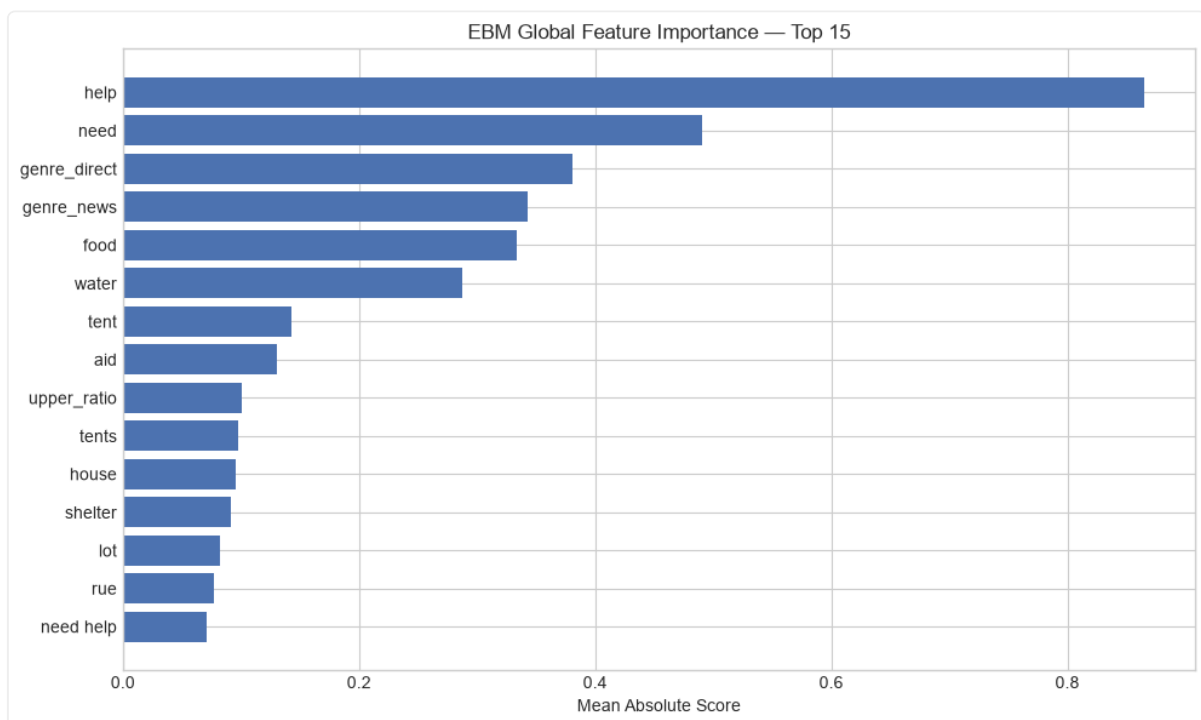
```
print("\nEBM Global Explanation:")
ebm_global = ebm.explain_global()

# Get top features from EBM
ebm_importances = ebm_global.data()['scores']
ebm_feat_names = ebm_global.data()['names']

# Plot top 15 EBM features
top_n = 15
idx_sorted = np.argsort(ebm_importances)[-top_n:]
fig, ax = plt.subplots(figsize=(10, 6))
ax.barh([ebm_feat_names[i] for i in idx_sorted], [ebm_importances[i] for i in idx_sorted],
        color='#4C72B0')
ax.set_title('EBM Global Feature Importance — Top 15')
ax.set_xlabel('Mean Absolute Score')
plt.tight_layout()
plt.show()
```

Output

EBM Global Explanation:



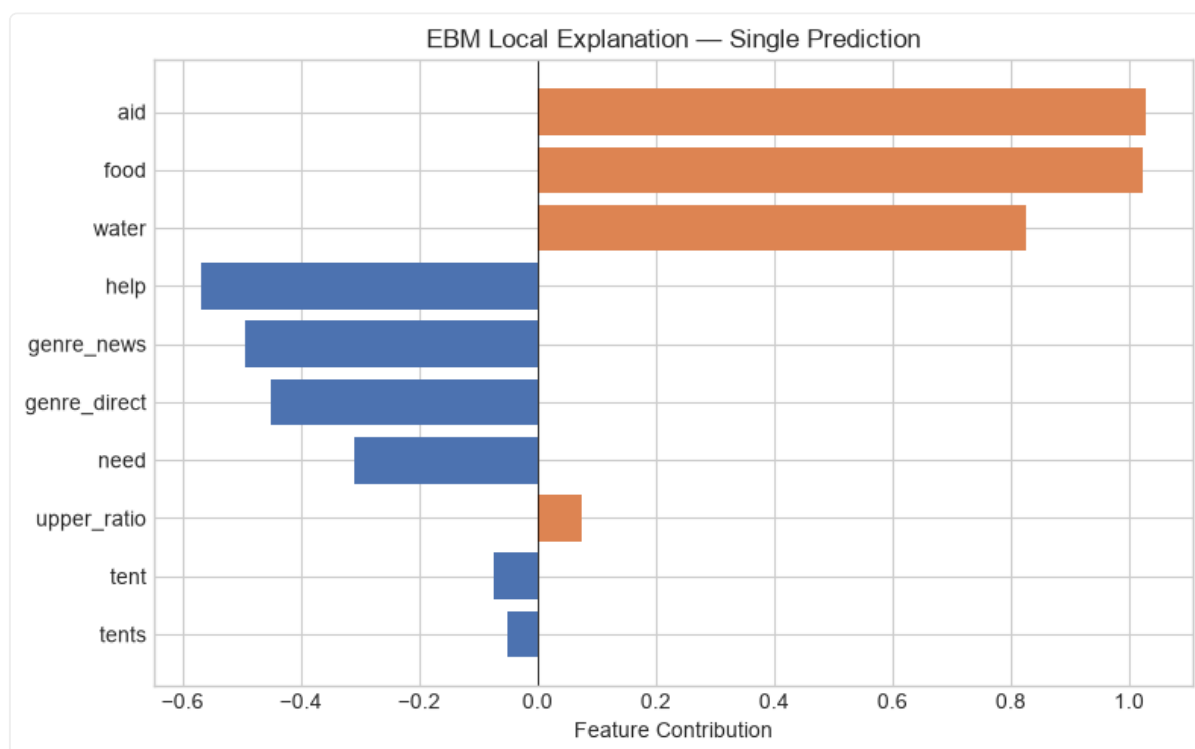
Code Input

```
print("\nEBM Local Explanation:")
ebm_local = ebm.explain_local(X_te_ebm[sample_idx:sample_idx+1], np.array([y_test[sample_idx]]))
local_data = ebm_local.data(0)
local_scores = local_data['scores']
local_names = local_data['names']

# Plot top contributing features for this prediction
top_local = min(10, len(local_scores))
idx_local = np.argsort(np.abs(local_scores))[-top_local:]
fig, ax = plt.subplots(figsize=(8, 5))
colors = ['#DD8452' if local_scores[i] > 0 else '#4C72B0' for i in idx_local]
ax.barh([local_names[i] for i in idx_local], [local_scores[i] for i in idx_local], color=colors)
ax.set_title('EBM Local Explanation — Single Prediction')
ax.set_xlabel('Feature Contribution')
ax.axvline(x=0, color='black', linewidth=0.5)
plt.tight_layout()
plt.show()
```

Output

EBM Local Explanation:



Ethics & Responsible Use

Concern	Discussion
Bias/Fairness	Training data is English-heavy; non-English speakers may be under-classified
Privacy	Dataset is anonymised public messages; no PII included
False Negatives	Missing a real emergency request is costly — model should be tuned for high recall
False Positives	Flagging non-emergencies wastes responder time but is less harmful
Human Oversight	Model should assist, not replace, trained emergency dispatchers
Domain Shift	Performance may degrade on disaster types not in training data
Deployment Limits	Not suitable as sole decision-maker; needs retraining for new disaster contexts

5. Live Demo — Predict on a Synthetic Record

Code Input

```
def predict_message(message, model=xgb_model, model_name='XGBoost'):
    """Predict whether a message is a request for help."""
    # Extract TF-IDF features
    tfidf_feat = tfidf.transform([message])

    # Extract metadata features
    meta = pd.DataFrame([
        'msg_len': len(message),
        'word_count': len(message.split()),
        'genre_direct': 1, # assume direct message
        'genre_news': 0,
        'genre_social': 0,
        'has_question': int('?' in message),
        'has_exclaim': int('!' in message),
        'upper_ratio': sum(1 for c in message if c.isupper()) / max(len(message), 1)
    ])
    meta_scaled = scaler.transform(meta)

    # Combine features
    X_new = hstack([tfidf_feat, csr_matrix(meta_scaled)])

    # Predict
    pred = model.predict(X_new)[0]
    proba = model.predict_proba(X_new)[0]

    print(f"\n{'='*60}")
    print(f"LIVE PREDICTION ({model_name})")
    print(f"{'='*60}")
    print(f"Message: {message}")
    print(f"Prediction: {'REQUEST FOR HELP' if pred == 1 else 'NOT A REQUEST'}")
    print(f"Confidence: {proba[pred]:.2%}")
    print(f"P(Not Request): {proba[0]:.4f} | P(Request): {proba[1]:.4f}")
    return pred, proba

# Test with synthetic emergency messages
test_messages = [
    "We need clean water urgently! Our village has been flooded and children are sick.",
    "Earthquake damage reported in the northern district. Buildings collapsed.",
    "Weather forecast shows clear skies for the weekend across the region.",
    "Please send medical supplies to the refugee camp. Many injured people here.",
]

for msg in test_messages:
    predict_message(msg)
```

Output

```
=====
LIVE PREDICTION (XGBoost)
=====
Message: We need clean water urgently! Our village has been flooded and children are sick.
Prediction: REQUEST FOR HELP
Confidence: 96.68%
P(Not Request): 0.0332 | P(Request): 0.9668

=====
LIVE PREDICTION (XGBoost)
=====
Message: Earthquake damage reported in the northern district. Buildings collapsed.
```

```
Prediction: REQUEST FOR HELP
Confidence: 76.80%
P(Not Request): 0.2320 | P(Request): 0.7680
```

```
=====
LIVE PREDICTION (XGBoost)
=====
```

```
Message: Weather forecast shows clear skies for the weekend across the region.
```

```
Prediction: REQUEST FOR HELP
Confidence: 66.83%
P(Not Request): 0.3317 | P(Request): 0.6683
```

```
=====
LIVE PREDICTION (XGBoost)
=====
```

```
Message: Please send medical supplies to the refugee camp. Many injured people here.
```

```
Prediction: REQUEST FOR HELP
Confidence: 84.36%
P(Not Request): 0.1564 | P(Request): 0.8436
```

Code Input

```

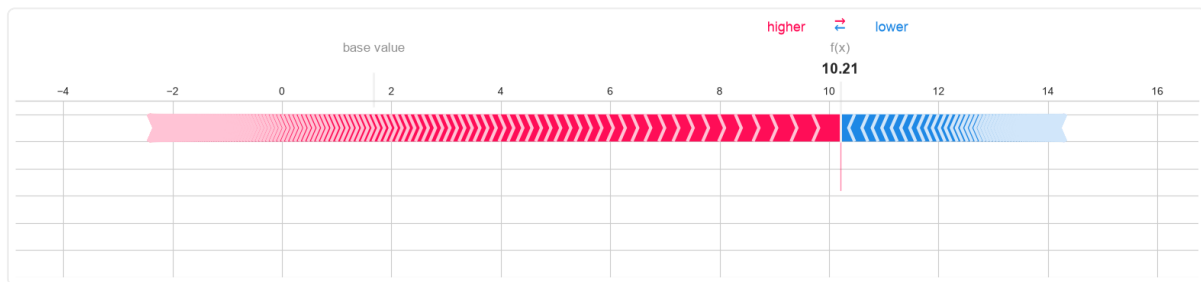
print("\nSHAP Explanation for synthetic message:")
test_msg = test_messages[0]
tfidf_feat = tfidf.transform([test_msg])
meta = pd.DataFrame([
    'msg_len': len(test_msg), 'word_count': len(test_msg.split()),
    'genre_direct': 1, 'genre_news': 0, 'genre_social': 0,
    'has_question': int('? ' in test_msg), 'has_exclaim': int('!' in test_msg),
    'upper_ratio': sum(1 for c in test_msg if c.isupper()) / max(len(test_msg), 1)
])
meta_scaled = scaler.transform(meta)
X_demo = hstack([tfidf_feat, csr_matrix(meta_scaled)]).toarray()

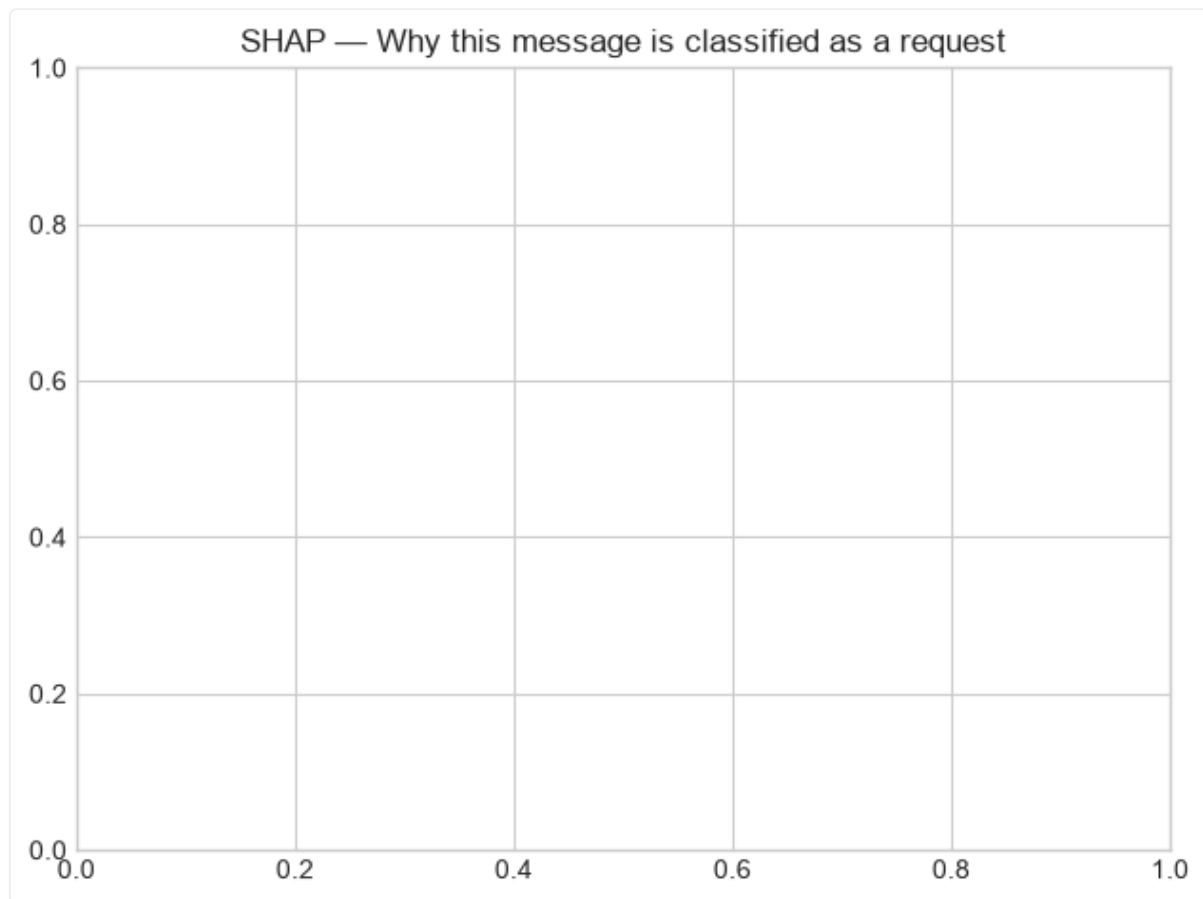
shap_vals_demo = explainer.shap_values(X_demo)
shap.force_plot(explainer.expected_value, shap_vals_demo[0], feature_names=all_feature_names,
matplotlib=True)
plt.title('SHAP – Why this message is classified as a request')
plt.tight_layout()
plt.show()

```

Output

SHAP Explanation for synthetic message:



**Code Input**

```
joblib.dump({
    'tfidf': tfidf,
    'scaler': scaler,
    'xgb_model': xgb_model,
    'ebm_model': ebm_model,
    'feature_names': all_feature_names
}, 'crisis_pipeline.joblib')
print("Pipeline saved to crisis_pipeline.joblib")
```

Output

```
Pipeline saved to crisis_pipeline.joblib
```

Summary

- **Baseline (Logistic Regression)** provides a solid starting point
- **Random Forest (Bagging)** improves via variance reduction
- **XGBoost (Boosting)** typically achieves best F1/AUC via sequential error correction
- **EBM (Glass-Box)** offers comparable accuracy with full interpretability
- **Stacking & Voting** combine diverse models for robust predictions
- **SHAP & EBM explanations** reveal which words and features drive predictions
- Model should augment human dispatchers, not replace them

```
print("\n" + "="*60)
```

```
print("PIPELINE COMPLETE")
```

```
print("="*60)
```